# Trabalho Prático 1

Arthur Mariano Maioli, Pedro Borsatto, Theo Duarte Lara

20 de outubro de 2026

## 1 Introdução

Diversos aplicativos de entrega - como o *IFood* ou o *99Pop* - precisam detectar os entregadores disponíveis dado um restaurante específico, para assim completar um pedido. No presente documento, apresentamos um sistema de geolocalização de restaurantes, em que após um estabelecimento ser localizado, é possível verificar quais entregadores estão mais próximos (Em que o conjunto desses entregadores é sintético, gerado através de uma distribuição normal).

O trabalho foi feito com o objetivo de testar o conhecimento dos alunos sobre algoritmos e estruturas de dados que favorecem indexação rápida para uma base de dados (Como a Trie). A seção de Análise Experimental explicita a melhora assintótica ao aplicar tais conhecimentos.

## 2 Preprocessamento da base de dados

Para fazer a geolocalização dos resturantes e bares, é preciso utilizar dados de latitude e longitude de cada estabelecimento, para assim fazer seu respectivo *Geohashing*. Como a base de dados incialmente disponibilizada carecia dessas informações geográficas, foi necessário cruzar as informações com a base de dados de Atividades Econômicas da PBH.

## 3 Simulação e Posicionamento de Entregadores

Para avaliar o sistema em um cenário próximo ao de plataformas reais de entrega, foi implementado um simulador capaz de gerar posições sintéticas para uma frota de $N$ entregadores na cidade de Belo Horizonte.

O processo é dividido em duas etapas: a geração probabilística das coordenadas em torno dos restaurantes e a projeção desses pontos para a malha viária da cidade.

### 3.1 Geração por Distribuição Normal

Em aplicações reais, os entregadores não se distribuem de maneira uniforme pelo território, mas tendem a se concentrar em regiões com maior oferta de pedidos.

Por esse motivo, o posicionamento inicial utiliza uma distribuição normal centrada nas coordenadas de cada restaurante da base de dados.

Considere $N$ a quantidade total de entregadores desejada e $R$ o número de restaurantes cadastrados. Os entregadores são divididos igualmente entre os estabelecimentos, atribuindo-se $\lfloor N/R \rfloor$ entregadores para cada restaurante e distribuindo-se o restante, $N \bmod R$, de forma unitária entre os primeiros registros.

Para cada restaurante localizado em $(\text{lat}_r, \text{lon}_r)$, as coordenadas dos entregadores são sorteadas a partir de um desvio padrão $\sigma_{\text{km}}$ (definido no projeto como 1,5 km).

Como as coordenadas estão em graus, utiliza-se a aproximação de que 1° de latitude corresponde a cerca de 111 km:

$$\sigma_{\text{lat}} = \frac{\sigma_{\text{km}}}{111}.$$

Na longitude, a distância correspondente a um grau diminui à medida que nos afastamos da linha do Equador. Para compensar essa distorção esférica, o desvio padrão longitudinal é ajustado pelo cosseno da latitude do restaurante:

$$\sigma_{\text{lon}} = \frac{\sigma_{\text{lat}}}{\cos(\text{lat}_r)}.$$

As coordenadas finais de cada entregador são então amostradas independentemente segundo:

$$\text{lat} \sim \mathcal{N}(\text{lat}_r, \sigma^2_{\text{lat}}), \qquad \text{lon} \sim \mathcal{N}(\text{lon}_r, \sigma^2_{\text{lon}}).$$

Pelas propriedades da distribuição normal, cerca de 68% dos entregadores gerados ficam a até $1\sigma$ (1,5 km) do restaurante de origem e aproximadamente 95% a até $2\sigma$ (3,0 km), formando maior densidade ao redor dos bares.

### 3.1.1 Análise de Complexidade

Considere $R$ a quantidade de restaurantes na base e $N$ o número total de entregadores gerados.

O algoritmo percorre a lista de restaurantes uma única vez para definir a quantidade de entregadores de cada centro e realiza o sorteio das $N$ coordenadas em tempo constante por entregador.

Portanto, a complexidade temporal da geração é:

$$\boxed{T_{\text{geracao}}(R, N) = O(R + N)}.$$

O espaço adicional necessário para armazenar o dicionário com os identificadores e coordenadas dos entregadores gerados é proporcional ao tamanho da frota:

$$\boxed{S_{\text{geracao}}(N) = O(N)}.$$

## 3.2 Projeção na Malha Viária

Como o sorteio pela distribuição normal é puramente matemático, parte das coordenadas geradas deve cair em locais inacessíveis para veículos, como o interior de quarteirões, edificações, parques ou corpos d'água.

Para corrigir esse comportamento e garantir que todos os entregadores apareçam sobre ruas transitáveis, foi implementada uma etapa de projeção na malha viária de Belo Horizonte utilizando a biblioteca `OSMnx`, que extrai os dados do *OpenStreetMap*.

A rede viária é modelada como um grafo direcionado $G = (V, E)$, em que os vértices $V$ representam interseções ou pontos das vias e as arestas $E$ representam os trechos de rua transitáveis por veículos (`network_type='drive'`).

Para evitar o custo de baixar o grafo a cada execução ou consulta, a malha viária é salva localmente em disco no formato `GraphML` na primeira execução e mantida em memória por meio de cache (`@lru_cache`).

Dado o conjunto de coordenadas sorteadas na etapa anterior, o algoritmo localiza, para cada entregador, o vértice $v \in V$ mais próximo na rede viária através da função `nearest_nodes` do `OSMnx`.

Em seguida, a posição original do entregador é substituída pelas coordenadas geográficas $(\text{lat}_v, \text{lon}_v)$ do nó viário correspondente.

### 3.2.1 Análise de Complexidade

Considere $V$ o número de vértices do grafo viário já carregado em memória e $N$ a quantidade de entregadores a serem projetados.

A busca pelos nós mais próximos utiliza uma árvore espacial ($k$-d tree) construída sobre os $V$ vértices do grafo.

A construção dessa estrutura auxiliar sobre os nós da rede viária possui custo $O(V \log V)$, enquanto a busca do nó mais próximo para cada um dos $N$ entregadores exige tempo $O(\log V)$ no caso médio.

Assim, o custo temporal da etapa de projeção é:

$$\boxed{T_{\text{projecao}}(V, N) = O((V + N) \log V)}.$$

O armazenamento das novas coordenadas projetadas para os entregadores requer espaço adicional:

$$\boxed{S_{\text{projecao}}(N) = O(N)},$$

além do espaço $O(V + E)$ ocupado pelo grafo da malha viária mantido em cache.

# 4 Indexação Geoespacial por Geohash

O Geohash é uma técnica de codificação que transforma coordenadas geográficas, latitude e longitude, em uma sequência de caracteres. Essa representação per-

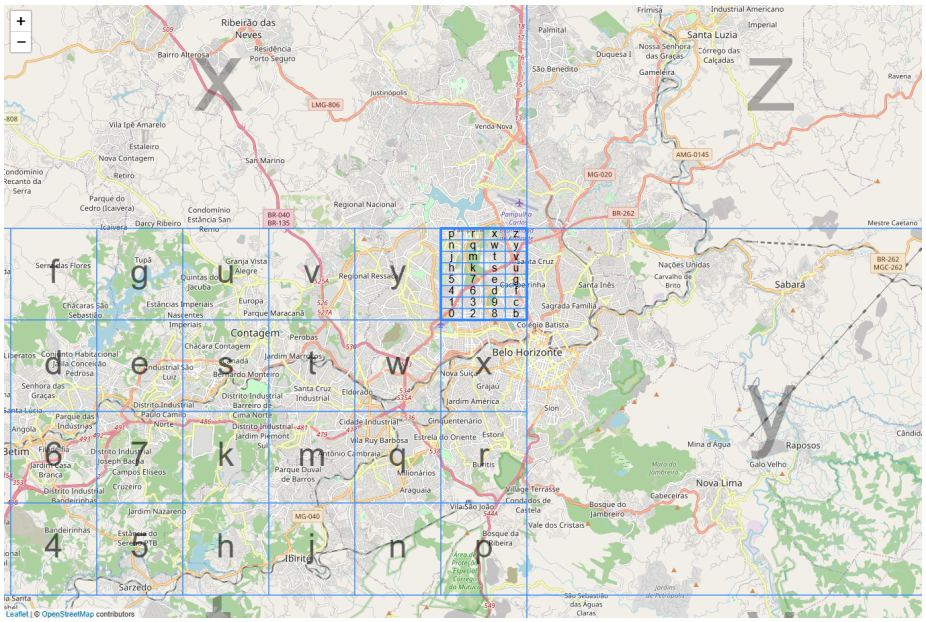



Figure 1

mite organizar pontos geográficos de maneira hierárquica, facilitando consultas por regiões.

O algoritmo funciona por meio de subdivisões sucessivas dos intervalos de latitude e longitude. Inicialmente, consideram-se os intervalos globais de longitude, $[-180°, 180°]$, e latitude, $[-90°, 90°]$.

A cada iteração, um desses intervalos é dividido ao meio, e um bit é produzido de acordo com a posição da coordenada em relação ao ponto médio. As subdivisões alternam entre longitude e latitude, começando pela longitude.

A sequência binária resultante é agrupada em blocos de cinco bits, convertidos em caracteres utilizando o alfabeto Base32 específico do Geohash:

`0123456789bcdefghjkmnpqrstuvwxyz`

Neste projeto adotamos uma precisão máxima de oito caracteres, equivalente a 40 subdivisões binárias. A precisão determina o tamanho das células geográficas representadas: quanto maior o Geohash, menor a região correspondente.

Uma propriedade fundamental dessa codificação é que Geohashes com prefixos comuns representam regiões contidas em uma mesma célula geográfica de maior dimensão. Essa característica possibilita a utilização de árvores de prefixos para indexar as posições dos entregadores.

### 4.1 Análise de Complexidade

Considere $p$ a quantidade de caracteres desejada para o Geohash. Como cada caractere representa cinco bits, o algoritmo realiza exatamente $5p$ subdivisões binárias.

Cada subdivisão envolve um número constante de operações: cálculo do ponto médio, comparação e atualização dos limites.

Portanto, a complexidade temporal é:

$$T_{\text{geohash}}(p) = O(5p) = \boxed{O(p)}.$$

O armazenamento da sequência binária e da representação Base32 também requer espaço proporcional à precisão:

$$S_{\text{geohash}}(p) = \boxed{O(p)}.$$

Como a precisão máxima utilizada no projeto é de oito caracteres, o custo de codificação é constante em relação à quantidade de entregadores cadastrados:

$$T_{\text{geohash}} = O(1),$$

considerando $p \leq 8$.

## 5 Indexação por Trie Compacta

Para armazenar e consultar os Geohashes dos entregadores, foi implementada uma Trie compacta.

Diferentemente de uma Trie convencional, em que cada aresta corresponde a um único caractere, a Trie compacta permite armazenar sequências de caracteres em um mesmo nó.

Dessa forma, caminhos sem ramificações são comprimidos, reduzindo a quantidade de nós necessários.

Cada nó da estrutura contém três atributos principais:

- `prefix`: sequência de caracteres correspondente ao segmento armazenado no nó;
- `children`: dicionário que associa o primeiro caractere de cada ramificação ao respectivo nó filho;
- `deliverers`: lista dos entregadores associados ao Geohash representado pelo caminho até o nó.

## 5.1 Inserção

A inserção percorre a árvore comparando o Geohash do novo entregador com os prefixos armazenados nos nós.

Quando não existe uma ramificação compatível, um novo nó é criado contendo o restante do Geohash.

Quando o prefixo de um nó é inteiramente compartilhado com o Geohash inserido, a busca prossegue para os seus descendentes.

Caso os dois prefixos compartilhem apenas uma parte inicial, ocorre uma divisão do nó (*split*).

O prefixo comum passa a representar um nó intermediário, enquanto os segmentos divergentes são armazenados em ramificações distintas.

Essa operação preserva a compactação da estrutura, evitando a criação de nós intermediários desnecessários.

### 5.1.1 Análise de Complexidade

Considere $p$ o comprimento do Geohash inserido.

A implementação utiliza um dicionário para acessar diretamente o filho correspondente ao próximo caractere da chave.

O acesso ao dicionário possui custo esperado $O(1)$.

Durante a inserção, cada caractere do Geohash é examinado no máximo uma quantidade constante de vezes.

Quando ocorre um *split*, são realizadas operações de criação de nós e cópia de segmentos da string, cujo custo total é limitado por $O(p)$.

Assim, a complexidade temporal esperada de uma inserção é:

$$\boxed{T_{\text{insert}}(p) = O(p)}.$$

O espaço adicional utilizado por uma inserção também é limitado pelo comprimento da chave:

$$\boxed{S_{\text{insert}}(p) = O(p)}.$$

Para $N$ entregadores, todos com Geohashes de comprimento máximo $p$, a construção da Trie possui custo esperado:

$$\boxed{T_{\text{construcao}}(N, p) = O(Np)}.$$

O armazenamento total dos segmentos e das referências aos entregadores também é limitado por:

$$\boxed{S_{\text{Trie}}(N, p) = O(Np)}.$$

Como $p \leq 8$, a construção possui custo linear em relação à quantidade de entregadores:

$$T_{\text{construcao}}(N) = O(N).$$

## 5.2 Busca por Prefixo

A busca por prefixo permite recuperar todos os entregadores associados a uma determinada região geográfica.

O algoritmo percorre a Trie comparando o prefixo consultado com os segmentos armazenados nos nós.

Existem três situações principais:

1. O prefixo procurado é completamente encontrado: todos os entregadores da subárvore correspondente são recuperados.
2. O segmento armazenado no nó é completamente encontrado, mas ainda restam caracteres da consulta: a busca continua nos descendentes.
3. Os prefixos divergem antes de uma correspondência completa: a busca é encerrada sem resultados.

Uma particularidade da Trie compacta é que o prefixo consultado pode terminar no interior do segmento armazenado em um nó.

Nesse caso, todos os entregadores de sua subárvore também pertencem à região procurada.

Após localizar a subárvore correspondente, uma busca em profundidade (*Depth-First Search*, DFS) percorre seus descendentes e reúne os entregadores encontrados.

### 5.2.1 Análise de Complexidade

Considere:

- $m$: comprimento do prefixo procurado;
- $v$: quantidade de nós visitados na coleta;
- $k$: quantidade de entregadores retornados.

A localização do prefixo exige examinar, no máximo, um número de caracteres proporcional a $m$, considerando também a comparação com o segmento final.

Como a precisão dos Geohashes é limitada , esse custo é $O(m)$ no contexto do projeto.

Após encontrar a região, a função recursiva de coleta visita todos os nós da subárvore e reúne suas listas de entregadores. O custo dessa etapa é $O(v + k)$, e portanto, a complexidade total da consulta é $O(m + v + k)$.

Em uma Trie compacta sem caminhos unários desnecessários, cada nó interno de ramificação possui pelo menos dois filhos.

Como cada subárvore terminal contém pelo menos um entregador, o número de nós visitados na coleta é proporcional ao número de resultados, para $k > 0$.

Assim, a complexidade pode ser expressa como:

$$\boxed{T_{\text{busca}}(m, k) = O(m + k)}.$$

Essa propriedade caracteriza uma busca sensível ao tamanho da saída: o custo depende do prefixo consultado e da quantidade de entregadores encontrados, e não diretamente do número total de entregadores armazenados na estrutura.

A lista de resultados requer espaço adicional:

$$\boxed{S_{\text{resultado}} = O(k)}.$$

Além disso, a implementação recursiva utiliza uma pilha de chamadas proporcional à altura da subárvore percorrida.

# 6 Busca Espacial de Entregadores

A busca espacial combina a indexação por Geohash e Trie compacta com uma etapa posterior de filtragem geométrica.

O objetivo é identificar os entregadores próximos ao restaurante selecionado, evitando a necessidade de calcular a distância entre o restaurante e todos os entregadores cadastrados.

## 6.1 Seleção das Regiões Candidatas

Inicialmente, as coordenadas do restaurante são convertidas em Geohash.

A consulta pode utilizar um prefixo de menor precisão que o Geohash completo, permitindo representar uma região geográfica mais ampla.

Entretanto, como o Geohash divide o espaço em células retangulares, pontos geograficamente próximos podem pertencer a células distintas, especialmente quando estão próximos às bordas.

Por isso, a busca deve considerar também as células adjacentes necessárias para cobrir o raio de atendimento.

Os prefixos correspondentes às regiões selecionadas são consultados na Trie, produzindo um conjunto de entregadores candidatos.

### 6.1.1 Análise de Complexidade

Considere $c$ a quantidade de células consultadas, $m$ o comprimento dos prefixos utilizados e $k_i$ a quantidade de entregadores recuperados na consulta da célula $i$.

Para cada célula, realiza-se uma busca por prefixo na Trie compacta.

O custo total das consultas é:

$$T_{\text{consultas}} = O\left(\sum_{i=1}^{c}(m + k_i)\right).$$

Quando todos os prefixos possuem o mesmo comprimento, essa expressão pode ser simplificada para:

$$\boxed{T_{\text{consultas}} = O\left(cm + \sum_{i=1}^{c} k_i\right)}.$$

Se as células consultadas não possuem sobreposição, cada entregador é recuperado no máximo uma vez.

Definindo $K$ como o número total de candidatos, obtém-se:

$$\boxed{T_{\text{consultas}} = O(cm + K)}.$$

Esse resultado não inclui o custo de determinar quais células devem ser consultadas, que depende do algoritmo de cobertura espacial adotado.

Caso as consultas possam produzir candidatos repetidos, também é necessário considerar o custo da deduplicação.

## 6.2 Filtragem por Distância

A busca por prefixo constitui uma etapa de pré-seleção espacial.

Como as células Geohash são retangulares, nem todos os entregadores recuperados estarão necessariamente dentro do raio de atendimento.

Assim, os candidatos passam por uma segunda etapa, na qual a distância entre cada entregador e o restaurante é calculada.

Somente os entregadores cuja distância seja menor ou igual ao raio definido pelo usuário são considerados aptos para o atendimento.

Essa combinação de indexação espacial e filtragem geométrica permite reduzir a quantidade de cálculos de distância, mantendo a precisão necessária para a consulta.

### 6.2.1 Análise de Complexidade

Considere $K$ a quantidade de entregadores candidatos recuperados na etapa anterior.

O cálculo da distância entre dois pontos geográficos possui custo constante, considerando operações aritméticas de custo unitário.

Como cada candidato precisa ser examinado uma vez, o custo da filtragem é:

$$\boxed{T_{\text{filtro}}(K) = O(K)}.$$

O armazenamento dos entregadores classificados também requer, no máximo:

$$\boxed{S_{\text{filtro}}(K) = O(K)}.$$

Essa etapa é necessária porque o pertencimento a uma célula Geohash não garante que o entregador esteja dentro do raio circular solicitado.

## 6.3 Complexidade Total da Busca Espacial

A consulta espacial envolve três etapas principais:

1. Codificação da localização do restaurante;
2. Busca dos candidatos nas células selecionadas;
3. Filtragem dos candidatos pela distância.

Seja $T_{\text{celulas}}$ o custo de determinar as $c$ células consultadas.
A complexidade total é:

$$T_{\text{espacial}} = O\left(p + T_{\text{celulas}} + cm + K\right).$$

Portanto:

$$\boxed{T_{\text{espacial}} = O(p + T_{\text{celulas}} + cm + K)}.$$

Como a precisão máxima é limitada a oito caracteres, $p$ e $m$ são limitados por constantes no projeto.
Consequentemente:

$$\boxed{T_{\text{espacial}} = O(T_{\text{celulas}} + c + K)}.$$

Se a geração das células candidatas possui custo $O(c)$, a expressão se reduz a:

$$\boxed{T_{\text{espacial}} = O(c + K)}.$$

Essa complexidade é vantajosa quando o número de entregadores candidatos é significativamente menor que o número total de entregadores cadastrados.

Entretanto, no pior caso, uma consulta pode recuperar todos os $N$ entregadores, resultando em custo $\Omega(N)$.

## 6.4 Integração dos Algoritmos

O processamento de uma consulta segue a sequência:

1. Obtenção das coordenadas do restaurante;
2. Codificação da posição em Geohash;
3. Determinação das células que devem ser consultadas;
4. Busca dos prefixos correspondentes na Trie compacta;
5. Reunião dos entregadores candidatos;
6. Filtragem dos candidatos pelo raio de atendimento;

7. Apresentação dos resultados na interface.

Dessa maneira, o Geohash realiza a discretização espacial, a Trie compacta organiza e recupera os candidatos por prefixo, e a filtragem geométrica determina quais entregadores atendem ao critério de proximidade.

# 7 Análise Experimental

Nesta seção, vamos avaliar o desempenho e a corretude do algoritmo proposto. Os experimentos estão divididos em duas etapas: (i) a validação espacial por meio do filtro de distância e (ii) a análise de escalabilidade e ganho de desempenho proporcionado pela árvore Trie compacta.

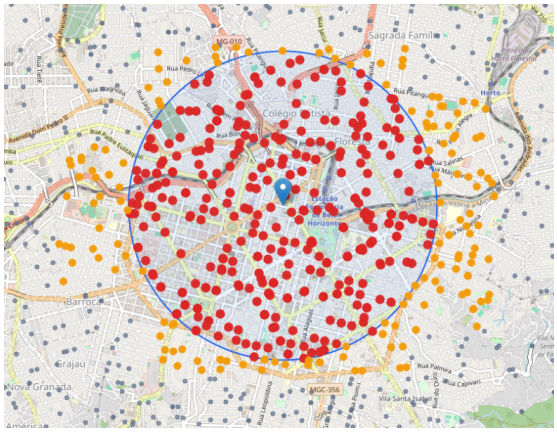
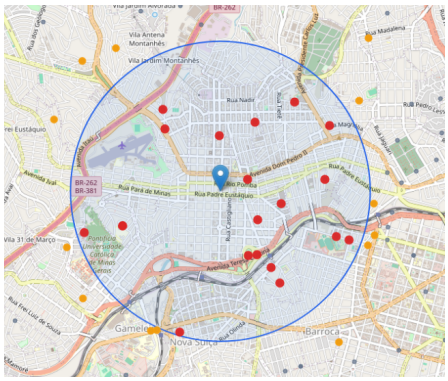


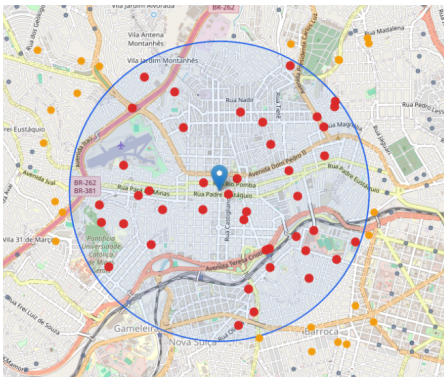

Figure 2: Recuperação de candidatos na Trie seguida pela filtragem de distância.

A Figura 2 ilustra uma consulta realizada a partir do restaurante *FOX LANCHES LTDA* com raio de busca de 2 km. A busca inicial na árvore Trie com poda espacial retorna 516 entregadores candidatos, correspondentes às células de Geohash que intersectam o raio estipulado. Contudo, como as células possuem formato retangular, parte de sua área pode passar a circunferência da busca. Aplicando o refinamento pelo cálculo da distância (fórmula de Haversine), identificamos que 354 entregadores (pontos vermelhos) estão efetivamente dentro do raio de 2 km, enquanto os 162 restantes (pontos laranjas) são falsos positivos descartados pelo filtro de distância.

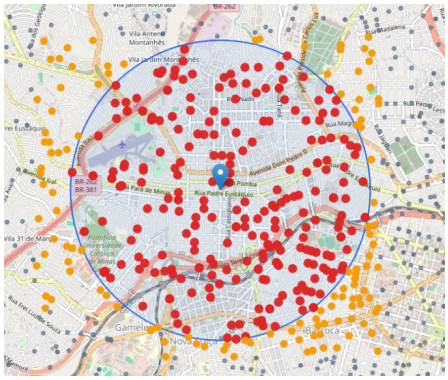
Em seguida, vamos avaliar o ganho de eficiência obtido pela pesquisa feita pela árvore Trie em comparação a uma busca linear dos entregadores. A Figura 3 apresenta o resultado visual de quatro cenários de teste com raio fixo de 2 km, variando o volume total da base em 500, 1.000, 5.000 e 10.000 entregadores. Os tempos de resposta registrados foram de 0,30 s, 0,41 s, 0,42 s e 0,57 s, respectivamente. A abordagem combina a recuperação de prefixos na Trie com a verificação exata de distância apenas para os candidatos retornados, resultando



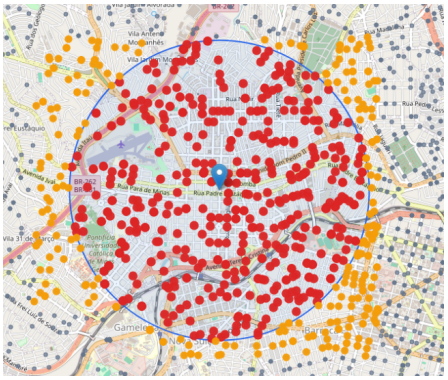
(a) 500 entregadores / 0.30s



(b) 1000 entregadores / 0.41s



(c) 5000 entregadores / 0.42s



(d) 10000 entregadores / 0.57s

Figure 3: Consultas espaciais com raio de 2 km variando o número total de entregadores.

em complexidade $O(m+k)$, onde $m$ está associado à navegação nos prefixos da árvore e $k$ ao número de candidatos avaliados. Com proposito de comparação, implementamos também a busca linear direta sobre toda a base de entregadores.

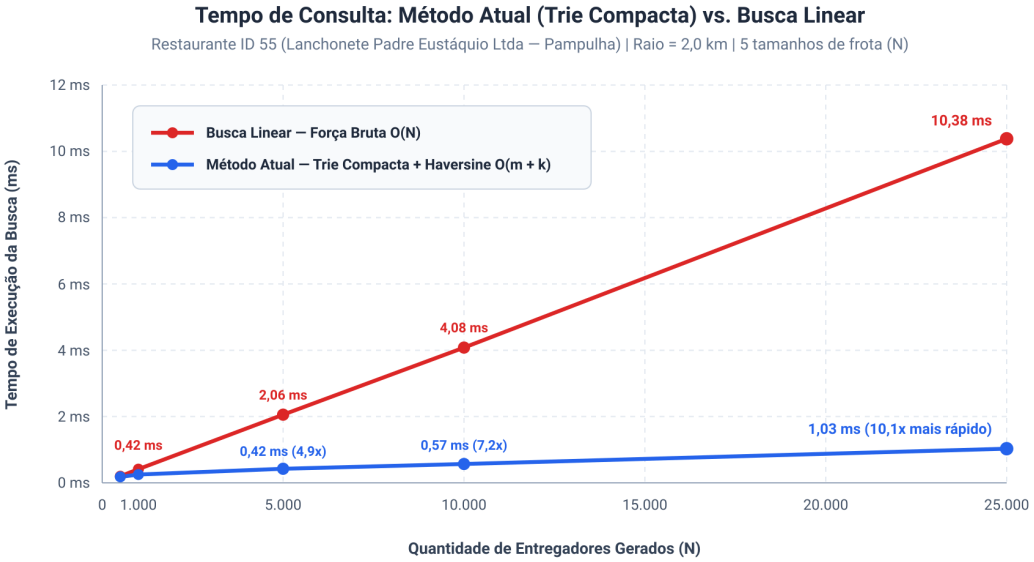



Figure 4: Comparação do tempo de execução entre a busca na Trie e a busca linear.

Por fim, a Figura 4 compara a escalabilidade da busca indexada pela Trie frente à varredura linear (foram executadas testes adicionais de testes além das ilustradas na Figura 3, omitidas visualmente para evitar poluição gráfica). Pode-se ver que o custo da busca na Trie cresce de forma significativamente mais suave conforme o volume de dados aumenta; para cenários com 25.000 entregadores, já se observa uma vantagem expressiva em tempo absoluto, com uma redução de aproximadamente 9 ms por consulta em relação à abordagem linear.

# 8 Conclusão

Neste trabalho, foi desenvolvido e avaliado um sistema de busca geoespacial para a localização de entregadores próximos a restaurantes na cidade de Belo Horizonte, combinando a codificação espacial via *Geohash* com a indexação em uma *Trie* compacta. Para tornar os testes que representam um cenário real de entregas, implementou-se também um módulo de geração sintética de entregadores baseado em distribuição normal centrada nos estabelecimentos, seguido pela projeção das coordenadas na malha viária real obtida via *OpenStreetMap*

(`OSMnx`) para deixar ainda mais realístico.

Do ponto de vista teórico e algorítmico, a utilização de *Geohashes* de precisão fixa permitiu transformar coordenadas bidimensionais contínuas em chaves lexicográficas hierárquicas em tempo $O(1)$. Junto com à *Trie* compacta, essa representação viabilizou buscas por prefixo sensíveis ao tamanho da saída, reduzindo o espaço de busca de toda a frota cadastrada ($N$) para apenas o subconjunto de entregadores candidatos ($K$) situados nas células que intersectam o raio de atendimento, atingindo complexidade $O(c + K)$ na consulta espacial. Além disso, a etapa complementar de filtragem geométrica pela distância de Haversine mostrou-se fundamental para eliminar falsos positivos decorrentes da geometria retangular das células do *Geohash*, garantindo que a busca retorne os entregadores que estão de fato no raio requisitado.

Os resultados experimentais confirmaram a eficácia e a escalabilidade da solução proposta em comparação à busca linear exaustiva. À medida que o número de entregadores aumentou — chegando a cenários com até 25.000 registros —, a abordagem indexada pela *Trie* compacta apresentou um crescimento de custo consideravelmente mais suave e reduções expressivas no tempo de resposta por consulta. Conclui-se, portanto, que a combinação de estruturas de indexação baseadas em prefixos com técnicas de discretização espacial permitiu uma solução eficiente, escalável e adequada para problemas de geolocalização em tempo real.